

LLM Prompting and Context Engineering for Bioinformatics

Master the art of effectively communicating with Large Language Models for
biological data analysis and research

PY4BIO 2025



Lecture Overview & Objectives

Learning Objectives

- ✓ Understand **LLM fundamentals** and their relevance to bioinformatics
- ✓ Master **prompting techniques** from basic to advanced
- ✓ Apply **context engineering** principles for optimal results
- ✓ Implement LLMs for **bioinformatics applications**
- ✓ Develop practical skills through **hands-on exercises**

 Duration: 0.5 Hours

Lecture Structure

- 1 Introduction to LLMs in Bioinformatics
- 2 Fundamentals of LLM Prompting
- 3 Advanced Prompting Techniques
- 4 Context Engineering Principles
- 5 Bioinformatics Applications
- 6 Hands-on Exercises & Best Practices

Introduction to LLMs & Their Relevance to Bioinformatics

🧠 What are LLMs?

- ⚡ **Large Language Models** are AI systems trained on vast text datasets
- 👤 Built on **transformer architecture** with billions of parameters
- ⚙️ Capable of understanding context, generating text, and performing complex reasoning
- 🔧 Examples: GPT-4, BERT, Claude, LLaMA, Gemini

↗️ Parameter count: 1B to 1T+

🔬 Why LLMs Matter in Bioinformatics

- 📊 Process **massive biological datasets** with pattern recognition
 - Analyze **DNA/RNA sequences** and predict genetic variations
- 🔍 Predict **protein structures** and functions
- 🚀 Accelerate **drug discovery** and target identification



Fundamentals of LLM Prompting: Basic Concepts

Key Terminology

-  **Prompt:** Input text given to an LLM to elicit a response
-  **Context Window:** Maximum tokens an LLM can process at once
-  **System Prompt:** Initial instructions that set model behavior
-  **Fine-tuning:** Adapting a pre-trained model for specific tasks
-  **Tokens:** Basic units of text processed by LLMs

 Clear instructions = Better results

Prompt Structure

-  Task Description
-  Context Information
-  Specific Instructions
-  Examples (if needed)
-  Output Format

```
"Analyze the following DNA sequence for potential protein-coding regions. Identify start and stop codons, and predict the amino acid sequence. Provide results in a tabular format with position, codon, and amino acid columns."
```

Prompting Techniques: Zero-shot and Few-shot

⚡ Zero-shot Prompting

- ✓ Model performs task **without examples**
- ✓ Relies on **pre-trained knowledge**
- ✓ Works well for **common tasks**

"Identify potential protein domains in this sequence: MKFLVNVALVFMVVYISYI..."



📄 Few-shot Prompting

- ✓ Model learns from **few examples** in prompt
- ✓ Helps with **complex or specialized tasks**
- ✓ More **reliable outputs** for domain-specific tasks

👍 Best For

- Simple classification
- Common tasks
- Quick prototyping

🗨️ Limitations

- Complex reasoning
- Domain-specific tasks
- Consistent formatting

Advanced Prompting Techniques

Chain-of-Thought (CoT)

- ✓ Guides LLM to show reasoning steps
- ✓ Improves **complex problem-solving**
- ✓ Enhances **accuracy** for multi-step tasks

"Analyze this protein sequence and explain your reasoning step-by-step to predict its function..."

 Improves accuracy by 30-50% for complex tasks

Retrieval Augmented Generation (RAG)

- ✓ Combines LLM with **external knowledge**
- ✓ Reduces **hallucinations** in responses
- ✓ Enables **up-to-date information** access

1 Query external knowledge base

2 Retrieve relevant documents

3 Augment prompt with context

4 Generate informed response

Context Engineering: Principles and Components

What is Context Engineering?

-  Evolution of prompt engineering
-  **Systematic process** of designing and optimizing instructions
-  Optimizes **information** provided to LLMs
-  Enables **consistent, reliable outputs**

” *Context engineering is the process of designing and optimizing instructions and relevant context for LLMs to perform tasks effectively.*

Key Components



System Prompts

High-level instructions that define model behavior and role



Relevant Knowledge

Domain-specific information and examples



Structured Inputs/Outputs

Defined formats for data exchange



Instructions



Historical Context



Tool Definitions



Memory Systems

LLM Applications in Bioinformatics: Overview

Key Application Areas

Genomics

DNA/RNA sequence analysis, variant calling, gene annotation

Protein Analysis

Structure prediction, function annotation, protein design

Drug Discovery

Target identification, molecule generation, interaction prediction

Literature Mining

Knowledge extraction, hypothesis generation, trend analysis

Specialized Bioinformatics LLMs

✓ ESM-1b

✓ DNABERT

✓ ProtBERT

✓ RNABERT

✓ ESMFold

✓ ProteinGPT

✓ ChemBERTa

Impact & Advantages

Accelerated Research

- Faster hypothesis generation
- Automated data analysis
- Reduced experimental cycles

Pattern Recognition

- Complex sequence relationships
- Hidden biological patterns
- Multi-omics integration

Research Growth



Bioinformatics LLM publications increased 425% from 2020-2024

70%

Improved accuracy in protein function prediction

10×

Faster variant annotation compared to traditional methods

Hands-on Examples and Exercises

<> Practical Prompting Examples

Gene Sequence Analysis

"Analyze the following DNA sequence and identify potential coding regions. Provide start/stop codon positions and predicted amino acid sequences: [sequence]"

- ✓ Use **specific delimiters** to separate input from instructions
- ✓ Request **structured output** format for easier parsing

Protein Structure Prediction

"Using the protein sequence below, predict secondary structure elements and potential functional domains. Provide reasoning for your predictions: [sequence]"

- ✓ Include **Chain-of-Thought** instruction for reasoning
- ✓ Specify **confidence levels** for predictions

Context Engineering Workflow

- 1 Define task requirements and constraints
- 2 Select appropriate prompting technique
- 3 Gather relevant domain knowledge
- 4 Structure prompt with clear components
- 5 Test and iterate with evaluation metrics

Exercise: RAG Implementation

- ✓ Create a **knowledge base** from protein function papers
- ✓ Design a **retrieval system** to find relevant context
- ✓ Build a **prompt template** that incorporates retrieved context

 Start with simple prompts, then gradually add complexity. Always measure performance improvements at each iteration.

Best Practices, Resources, and Conclusion

✓ Best Practices

💡 Effective Prompting

- ✓ Clear instructions with specific format requirements
- ✓ Provide **relevant context** within token limits
- ✓ Use **examples** for complex or specialized tasks
- ✓ Iterate and test with evaluation metrics

⚠ Common Pitfalls

- ✗ Ambiguous or **vague instructions**
- ✗ Overloading **context window**
- ✗ Ignoring **hallucinations** in responses

📖 Resources

📖 PromptingGuide.ai

🔬 BioBERT Models

<> OpenAI API Docs

📄 PubMed LLM Papers

📁 Bioinformatics Stack

🎥 Bioinformatics Tutorials

85%

Improved accuracy with proper context

3×

Faster analysis with effective prompting

Key Takeaway



Mastering LLM prompting and context engineering is now an essential skill for bioinformatics researchers to accelerate discoveries and enhance analysis capabilities

Specific Applications: Protein Analysis and Genomics

Protein Analysis

Structure Prediction

- ✓ 3D structure from amino acid sequences
- ✓ Fold **recognition** and classification
- ✓ Protein-protein **interaction** prediction

Key Models

✦ ESMFold

✦ ProtBERT

✦ ProteinGPT

✦ ESM-1b

"Analyze the following protein sequence and predict its secondary structure elements (alpha helices, beta sheets) and potential functional domains: [sequence]"

Genomics Applications

Sequence Analysis

- ✓ Gene **annotation** and function prediction
- ✓ **Variant calling** and pathogenicity assessment
- ✓ **Motif detection** and regulatory element identification

Key Models

✦ DNABERT

✦ DNABERT-2

✦ HyenaDNA

✦ GeneBERT

Real-world Impact

- 💡 DNABERT-2 improved cross-species genomic comparisons by 42% compared to traditional alignment methods

"Identify potential regulatory elements in this genomic region and predict their function based on known patterns: [sequence]"